

Optimization Algorithm

Scenario:

You are at a bazaar trying to buy a fruit basket for a party. There are many stalls, and each has a different price for similar fruit baskets. Your goal is

- Minimize the cost while still buying a good basket. That's optimization.

Let: n = the stall number

$f(n)$ = price of the basket at the stall

We want to find:

$$n_{\text{best}} = \arg \min_n f(n)$$

So we search for the n that gives the lowest $f(n)$.

That's exactly what optimization algorithms do.

Optimization Algorithm in Action:

❑ Manual Method (Brute Force):

You visit every n and keep the value of n and pick the lowest.

Accurate but slow.

❑ Gradient Descent (Like Smart Search):

You start at one stall $\rightarrow$ check the price slope (Is it going up or down?)

$\rightarrow$ Move in the cheaper direction, step by step.

Eventually, you reach the minimum price!

$$\begin{aligned} 0 &= 0 + 1 \times 0 = 10 \\ 0 &= 0 + 5 \times 0 = 50 \\ 0 &= 0 + 8 \times 0 = 80 \end{aligned}$$

Optimization algorithms are mathematical methods used to find the best solution to a problem, typically by minimizing a cost/loss function or maximizing a utility function.

Example: → In ML, we minimize loss.

→ In logistics, we minimize cost or maximize efficiency.

* Categories of Optimization Algorithms:

Categories	Examples	Used In
Gradient-Based	Gradient Descent, Newton's method	ML, DL, Economics
Heuristic/Metaheuristic	Genetic algo, PSO, simulated Annealing	AI, Engineering, scheduling
Linear Programming	Simplex, Interior Point	Business, Supply Chains
Constrained	Lagrange Multipliers, KKT Conditions	Engineering, Physics
Convex	Convex optimization, Quadratic Programming	ML, Control Systems.

Linear regression model using Gradient Descent.

Ex:

Dataset:

x	y
1	2
2	4
3	6

we want to learn $y = mx + b$;
So that we predict y as close as actual value

Step 1: Initialization:

start with random value; $m = 0$, $b = 0$

learning rate $\alpha = 0.01$

$$\hat{y}_1 = 0 \times 1 + 0 = 0$$

$$\hat{y}_2 = 0 \times 2 + 0 = 0$$

$$\hat{y}_3 = 0 \times 3 + 0 = 0$$

step 2: Error: (MSE)

$$e_1 = 2 - 0 = 2 \quad e_2 = 4 - 0 = 4 \quad e_3 = 6 - 0 = 6$$

step 3:

$$\frac{\partial \text{loss}}{\partial m} = -\frac{2}{3}(1 \times 2 + 2 \times 4 + 3 \times 6) = -18.67$$

$$\frac{\partial \text{loss}}{\partial b} = -\frac{2}{3}(2 + 4 + 6) = -8$$

step 4: update: $m := 0 - 0.01 \times (-18.67) = 0.1867$

$$b := 0 - 0.01 \times (-8) = 0.08$$

Iteration 2:

$$\hat{y}_1 = 0.1867 \times 1 + 0.08 = 0.2667$$

$$\hat{y}_2 = 0.1867 \times 2 + 0.08 = 0.4534$$

$$\hat{y}_3 = 0.1867 \times 3 + 0.08 = 0.6401$$

step 2:

$$e_1 = 2 - 0.2667 = 1.733 \quad ; \quad e_2 = 4 - 0.4534 = 3.5466 \quad ; \quad e_3 = 6 - 0.6401 = 5.3599$$

step 3:

$$\frac{\partial \text{loss}}{\partial m} = -\frac{2}{3}(1 \times 1.733 + 2 \times 3.5466 + 3 \times 5.3599) = -16.60$$

$$\frac{\partial \text{loss}}{\partial b} = -\frac{2}{3}(1.733 + 3.5466 + 5.3599) = -7.09$$

step 4:

$$\text{update: } m := 0.1867 + 0.01 \times (-16.60) = 0.3527$$

$$b := 0.08 + 0.01 \times (-7.09) = 0.1509$$

2

Confusion Matrix

Confusion matrix is a performance measurement tool for machine learning classification problems.

It shows how well a model's predictions match with the actual values, specially used for evaluating binary and multi-class classification models.

Type I Error/FP : Incorrectly predicted positive when it's actually negative

	Predicted Positive	Predicted Negative
Actual Pos.	TP	FN
Actual Neg.	FP	TN

Type II Error/FN : Predict negative but actually positive

Accuracy = $\frac{TP+TN}{TP+TN+FN+FP}$; Overall correctness of model

Precision = $\frac{TP}{TP+FP}$; How many predicted positive are actually positive

Recall = $\frac{TP}{TP+FN}$; How many actual positive are correctly predicted

F1 Score = $\frac{2 \times (\text{Precision} \times \text{Recall})}{(\text{Precision} + \text{Recall})}$; Harmonic mean of Recall and Precision

Example: $TP=70$; $TN=50$; $FP=10$ $FN=20$

$$\text{Accuracy} = \frac{70+50}{70+50+10+20} = 0.8$$

$$\text{Precision} = \frac{70}{70+10} = 0.875$$

$$\text{Recall} = \frac{70}{70+20} = 0.778$$

$$\text{F1 Score} = \frac{2 \times (0.875 \times 0.778)}{0.875 + 0.778} = 0.823$$

F1 Score

The F1 score is a performance metric used to evaluate the accuracy of a classification model, especially when the data is imbalanced.

$$F1 \text{ score} = \frac{2 \times (\text{Precision} \times \text{Recall})}{\text{Precision} + \text{Recall}}$$

- H. F1 score is the harmonic mean of precision & Recall.
- Range from 0 to 1
 - It balance the trade-off ~~false positive~~ between FP & FN.

When use:

- Use F1 score when FP & FN have similar cost.
- Especially useful in imbalanced datasets. Eg: fraud, spam, disease.

We can say,

$$\text{Precision} = \frac{\text{True Positive}}{\text{Total predicted positive}}$$

$$\text{Recall} = \frac{\text{True positive}}{\text{Total actual positive}}$$

Recall mean what percentage of the total actual positives were you able to correctly identify.

Red	A
Green	B
Blue	C
Red	D

MAP coloring

Map coloring is a classic graph coloring problem in computer science and mathematics.

The goal is to assign colors to regions on map so that no two ~~color~~ adjacent regions share the same color.

Why it matters:

- Prevent confusion in geographic maps
- Solving scheduling conflicts (ex: exam timetables, register allocation)
- Used in AI, Graph theory, Constraint satisfaction problem (esp)

Real life example: simplified map of 4 countries:

Adjacents:

A → B, C
B → A, D
C → A, D
D → B, C

* Two regions are adjacent if they share a common boundary.



Goal: Color each country so that no neighbouring country has the same color.

Soln: We can use 4 colors but the famous four color theorem says we never need more than 4 for any 2D map.

Country	Color
A	Red
B	Green
C	Blue
D	Red

No two countries have the same color.

(Ans) =

The Four Color Theorem states:

"Any map in a plane can be colored using at most four colors in such a way that no two adjacent regions share the same color."

* In 2D map, no matter how complicated - 4 colors are always enough - -

K - mean C Algorithm

K-means clustering is a method of vector quantization, originally from signal processing, that aims to partition n observations into k clusters in which each observation belongs to the cluster with the nearest mean, serving as a prototype of the cluster.

Example:

Given dataset:

	Height	weight
①	185	72
②	170	56
③	168	60
④	179	68
⑤	182	72
⑥	188	77
⑦	180	71
⑧	183	89
⑨	180	88
⑩	180	67
⑪	177	76

Initially set

$$R_1 = K_1 = 185, 72$$

$$R_2 = K_2 = 170, 56$$

$$K_1 = \{1, 4, 5, 6, 7, 8, 9, 10, 11\}$$
$$K_2 = \{2, 3\}$$

Generated output

We know,

$$\text{Euclidean Distance} = \sqrt{(X_0 - X_c)^2 + (Y_0 - Y_c)^2}$$

$$\text{ED for } R_3 \rightarrow K_1 = \sqrt{(168 - 185)^2 + (60 - 72)^2} = 20.80$$

$$\rightarrow K_2 = \sqrt{(168 - 170)^2 + (60 - 56)^2} = 4.47$$

New centroid Calculation:

here, $K_2 < K_1$ so, for $K_2 = \left(\frac{170+168}{2}, \frac{60+56}{2}\right) = (169, 58)$

$$\text{ED for } R_4 \rightarrow K_1 = \sqrt{(179 - 185)^2 + (68 - 72)^2} = 9.21$$
$$\rightarrow K_2 = \sqrt{(179 - 169)^2 + (68 - 58)^2} = 14.14$$

here, $K_1 < K_2$ so,

$$\text{for } K_1 = \left(\frac{185+179}{2}, \frac{72+68}{2}\right) = (182, 70)$$

do this for all Row

H K-Nearest Neighbor classification (KNN)

Given data $\Rightarrow$

r	Math.	CJ	Result
①	4	3	Fail
②	6	7	Pass
③	7	8	Pass
④	5	5	Fail
⑤	8	8	Pass

query for

$n(\text{math}=6, \text{CJ}=8)$ where

$k=3$.

Soln: We know:- $d = \sqrt{|x_{01} - x_{A1}|^2 + |x_{02} - x_{A2}|^2}$

① $\sqrt{(6-4)^2 + (8-3)^2} = 5.38$

② $\sqrt{(6-6)^2 + (8-7)^2} = ① \checkmark$

③ $\sqrt{(6-7)^2 + (8-8)^2} = ① \checkmark$

④ $\sqrt{(6-5)^2 + (8-5)^2} = 3.16$

⑤ $\sqrt{(6-8)^2 + (8-8)^2} = ② \checkmark$

As there the value of $k=3$ we choose the 3 minimum distance. 2, 3 and 5 row.

here, $\text{Pass} \rightarrow 3$

Fail $\rightarrow 0$

So $n \rightarrow \text{pass}$

37.0

Problem: Automated Malware Detection.

$$X_{\text{query}} = (14, 12)^T$$

Executable File	x_1	x_2	label/class(Y)
chrome.exe	2	12	Benign
vscode.exe	3	5	Benign
calc.exe	1	0	Benign
trojan_x.exe	15	18	malware
ransom.exe	18	14	malware

KNN module computes the ED $d(X_{\text{query}}, x_i)$ in $\mathbb{R}^2$:

$$d = \sqrt{(x_{1\text{query}} - x_{1i})^2 + (x_{2\text{query}} - x_{2i})^2}$$

1. (2, 12) : $\sqrt{(14-2)^2 + (12-12)^2} = 12.$

2. (3, 5) : $\sqrt{(14-3)^2 + (12-5)^2} = 13.03$

3. (1, 0) : $\sqrt{(14-1)^2 + (12-0)^2} = 17.69$

4. (15, 18) : $\sqrt{(14-15)^2 + (12-18)^2} = 6.08$

5. (18, 14) : $\sqrt{(14-18)^2 + (12-14)^2} = 4.47$

Sorting distance yield the top $k=3$ nearest instances in the feature space:

1. ransom.exe (4.47) $\rightarrow$ Malware

2. trojan_x.exe (6.08) $\rightarrow$ Malware

3. chrome.exe (12.00) $\rightarrow$ Benign

Majority Vote & system Action : Malware votes - 2
Benign votes - 1

Output: unknow.process.exe is classified as Malware. The antivirus kills the process thread and flags the file for quarantine.

Heuristic Algorithms

To solve a problem, we can use ~~direct~~ two approach

Approach A: The Brute-Force Method (Exact Algorithm)

Approach B: The smart shortcut (Heuristic Method)

Heuristics use intuition rules of thumb.

Heuristic algorithm define as - A problem-solving framework designed to find a "good enough" solution rapidly when classic exact algorithms are too slow.

Properties:

- ❑ Non-Optimal: Does not guarantee finding the absolute best solution.
- ❑ Efficient: Drastically cut down execution time from exponential time to polynomial time.
- ❑ Rule-of-Thumb Based: Uses domain-specific information or heuristics to guide search.

* Mathematical Example: The Knapsack Problem:

Bag Capacity 10 kg; Available weight Items

A $\rightarrow$ W-6 V-\$60

B $\rightarrow$ W-5 V-\$40

C $\rightarrow$ W-5 V-\$40

1. Greedy Heuristic Approach

• Item A Density = $\frac{60}{6} = 10 \$ / kg$ Item B $\rightarrow$ D = $\frac{40}{5} = 8 \$ / kg$

Item C $\rightarrow$ D = $\frac{40}{5} = 8 \$ / kg$; Select Item A first

Remaining capacity = $10 - 6 = 4$ Neither B nor C fits in 4 kg.

Heuristic Result: Total value = \$60

2. Exact solution:

Checking all combination and find out exact method.
But required testing all combination.

Exact Algo (DP)	Heuristic Algo (Greedy Approach)
Optimality — 100% optimal	Near-optimal / sub-optimal
Time Complexity — often exponential $O(2^n)$ / or $O(n!)$	Usually polynomial $O(n \log n)$ or $O(n)$
Scalability — Fails on large datasets	scales well to massive datasets
Use Case — Critical system Ex - financial, medical	Real time / Large-scale

Real-world application of heuristic —

① Route optimization - Google maps / A* search

② Game AI - Chess / Checkers.

③ Traveling salesperson Problem (TSP)

→ Best and good enough is usually better than perfect and infinite.

It especially useful when:

① The problem space is huge

② An exact solution is too slow or impossible to compute in reasonable time.

③ An approximate solution is acceptable.

Heuristic function in AI :-

1. Misplaced tiles - h_1

$$h_1 = 8$$

7	2	4
5	-	6
8	3	1

Start

-	1	2
3	4	5
6	7	8

Goal

2. Manhattan Distance - h_2

$$h_2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$$

3. Total heuristic value $h(n)$

combine both metrics together:

$$h(n) = h_1 + h_2 = 8 + 18 = 26$$

4 Difference between algorithm and a heuristic :-

⇒ An algorithm is a step-wise procedure for solving a specific problem in a finite number of steps.

⇒ A heuristic is an educated guess which serves as a guide for subsequent explorations.

Aspect	Algorithm	Heuristic
Definition	A step-by-step, well-defined procedure -	A rule of thumb or shortcut
% of solution	Yes	Not necessarily
Approach	Exhaustive, systematic and precise	Approximate, experience-based, used intuition
Speed	Can be slow for complex problems (NP-hard prob)	Usually faster because it avoids exploring every possibility
Flexibility	Work only for specific problem it was designed for	can often be adapted to many problem in the same domain
Example	merge sort	Sorting book by size

Are algorithms better than heuristics?

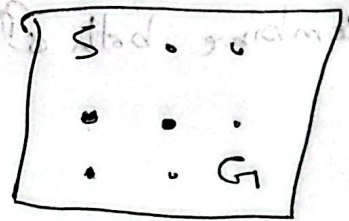
Ans: While algorithms provide step-by-step procedures that can guarantee solutions, heuristics are faster and provide shortcuts for getting to solutions, though this has the potential to cause errors.

A* search with example:

You are in a small 3×3 grid. You want to go from Start to Goal. You can only move up, down, left, right.

Coordinates: $S = (0,0)$

$G = (2,2)$



Sol:

We know,

A* use $f(n) = g(n) + h(n)$

$g(n)$ = steps from start so far

$h(n)$ = estimate steps to goal

walkthrough:

start at $(0,0)$; $g=0$; $h=|2-0| + |2-0| = 4$
 $f = 0 + 4 = 4$

Neighbors:
 $(1,0)$ $g=1$ $h=3$, $f=4$
 $(0,1)$ $g=1$ $h=3$, $f=4$

Pick one with smallest f , say $(1,0)$

Neighbors:
 $(2,0)$: $g=2$, $h=2$ $f=4$
 $(1,1)$: $g=2$ $h=2$ $f=4$

keep picking smallest f .

Eventually path chosen: $(0,0) \rightarrow (1,0) \rightarrow (2,0) \rightarrow (2,1) \rightarrow (2,2)$

P.T.O

shortest path found in 4 moves, and we didn't check every single cell blindly - the heuristic guide us toward the goal. $\approx$